CLRS 기반 · 구현 포함 한국어 학습 노트

그래프부터 위상정렬까지

이 문서는 *Introduction to Algorithms, 4th Edition*의 그래프 기초, BFS, DFS, 위상정렬을 재혁님 공부용으로 다시 풀어쓴 자료입니다. 이번 버전은 개념 설명만이 아니라 **Python 구현 코드, 실행 예시, 어떻게 쓰는지**까지 한 번에 보이도록 구성했습니다.

1. 그래프를 먼저 감으로 잡기

그래프는 **정점(vertex)**과 **간선(edge)**으로 이루어집니다. 정점은 대상 하나, 간선은 두 대상 사이의 연결입니다.

- 정점 예시: 사람, 도시, 과목, 작업
- 간선 예시: 친구 관계, 길, 선수 과목 관계, 작업 순서
- 무방향 그래프: 연결만 중요함
- 방향 그래프: 방향까지 중요함

> 그래프를 배우는 핵심은 두 가지입니다. **어떻게 저장할까?** 그리고 **어떤 순서로 탐색할까?**

2. 그래프를 코드로 어떻게 저장하나

2-1. 인접 리스트

가장 많이 쓰는 형태입니다. 각 정점마다 연결된 이웃 정점들을 리스트로 저장합니다.

```
graph = {
    1: [2, 3],
    2: [4],
    3: [4, 5],
    4: [1],
    5: [1]
}
```

예를 들어 `1: [2, 3]`은 "1번 정점에서 2번과 3번으로 갈 수 있다"는 뜻입니다.

- 장점: 메모리를 비교적 적게 씀
- 장점: 탐색할 때 연결된 정점들을 바로 볼 수 있음
- 단점: 두 정점이 직접 연결됐는지 즉시 확인하려면 리스트를 봐야 함

2-2. 인접 행렬

표처럼 저장하는 방식입니다. 정점이 4개라면 4 x 4 표를 만들고, 연결 여부를 0과 1로 표시합니다.

```
graph =
  [0, 1, 1, 0]
  [0, 0, 0, 1]
  [0, 0, 0, 1]
  [0, 0, 0, 0]
]
```

- 장점: 연결 여부를 바로 알 수 있음
- 단점: 정점 수가 커지면 메모리를 많이 씀

방법	언제 자주 쓰나	특징
인접 리스트	알고리즘 문제 대부분	간선이 적을 때 효율적
인접 행렬	정점 수가 작고 연결 여부를 자주 확인할 때	표 형태라 직관적이지만 메모리 사용량이 큼

3. BFS: 가까운 것부터 넓게 보기

BFS(Breadth-First Search)는 시작점에서 가까운 정점부터 차례대로 방문하는 탐색입니다. 그래서 보통 **큐(queue)**를 사용합니다.

왜 큐를 쓰나

먼저 들어온 정점을 먼저 처리해야 "가까운 거리 순서"가 유지되기 때문입니다.

동작 순서

1. 시작 정점을 큐에 넣는다.
2. 큐에서 하나 꺼낸다.
3. 그 정점과 연결된 이웃 중 아직 방문 안 한 정점을 큐에 넣는다.
4. 큐가 빌 때까지 반복한다.

BFS 구현 코드

```
from collections import deque

def bfs(graph, start):
    visited = set([start])
    queue = deque([start])
    order = []

    while queue:
        now = queue.popleft()
        order.append(now)

        for nxt in graph[now]:
            if nxt not in visited:
                visited.add(nxt)
                queue.append(nxt)

    return order
```

코드 해설

- `visited`: 이미 방문한 정점을 저장합니다.
- `queue`: 다음에 방문할 정점들을 순서대로 저장합니다.
- `order`: 실제 방문 순서를 기록합니다.
- `popleft()`: 큐의 맨 앞 원소를 꺼냅니다.

사용 예시

```
graph = {
    1: [2, 3]
```

```
    2: [4],
    3: [4, 5],
    4: [],
    5: []
}

print(bfs(graph, 1))
```

```
출력
[1, 2, 3, 4, 5]
```

왜 이렇게 나오냐면, 1에서 먼저 2와 3을 보고, 그 다음 거리의 정점들인 4와 5를 보기 때문입니다.

BFS가 특히 강한 문제

- 미로에서 최소 칸 수 구하기
- 가중치 없는 그래프의 최단 거리 구하기
- 몇 단계 만에 도달하는지 계산하기

> BFS는 "가까운 순서"가 중요할 때 잘 맞습니다. 최단 거리 느낌이 보이면 먼저 떠올려보면 좋습니다.

4. DFS: 한 길을 끝까지 가보기

DFS(Depth-First Search)는 한 방향으로 깊게 들어갈 수 있을 때까지 들어간 뒤, 막히면 돌아와서 다른 길을 찾는 탐색입니다.

왜 DFS가 중요하나

- 그래프 구조를 분석할 때 강합니다.
- 사이클 판별, 연결 요소, 위상정렬 같은 주제와 많이 연결됩니다.

재귀로 구현한 DFS

```
def dfs(graph, start, visited=None, order=None):
    if visited is None:
        visited = set()
    if order is None:
        order = []

    visited.add(start)
    order.append(start)

    for nxt in graph[start]:
        if nxt not in visited:
            dfs(graph, nxt, visited, order)

    return order
```

코드 해설

- 현재 정점을 방문 처리합니다.
- 현재 정점을 방문 순서에 넣습니다.
- 연결된 다음 정점으로 재귀 호출합니다.
- 더 갈 곳이 없으면 함수가 끝나면서 자동으로 되돌아옵니다.

사용 예시

```
graph = {
    1: [2, 3],
    2: [4],
    3: [5],
    4: [],
    5: []
}

print(dfs(graph, 1))
```

```
출력
[1, 2, 4, 3, 5]
```

1에서 시작해서 먼저 2로 가고, 2에서 4까지 끝까지 들어간 뒤, 그다음에 돌아와서 3과 5를 보는 흐름입니다.

스택으로 구현하면 어떤 느낌인가

재귀는 내부적으로 함수 호출 스택을 쓰고 있습니다. 그래서 직접 스택을 만들어도 비슷한 DFS를 구현할 수 있습니다. 다만 처음에는 재귀 버전이 흐름을 이해하기 더 쉽습니다.

5. BFS와 DFS를 비교해서 정리

구분	BFS	DFS
탐색 방식	가까운 곳부터 넓게	한 길을 깊게
주요 자료구조	큐	재귀 또는 스택
잘 맞는 문제	최단 거리, 단계 계산	구조 분석, 사이클, 위상정렬
시간복잡도	O(V + E)	O(V + E)

6. 위상정렬: 선행 작업을 먼저 두는 순서

위상정렬(topological sort)은 **방향 비순환 그래프(DAG)**에서만 가능한 정렬입니다. 간선 `u → v`가 있다면, 결과 순서에서는 반드시 `u`가 `v`보다 앞에 있어야 합니다.

쉽게 말하면 "먼저 해야 할 일을 먼저 배치하는 순서"입니다.

- 예: 자료구조를 먼저 배우고 알고리즘을 배운다
- 예: 설계를 먼저 하고 구현을 한다

왜 사이클이 있으면 안 되나

`A → B → C → A`처럼 사이클이 있으면 A가 먼저, B가 먼저, C가 먼저여야 하는 모순이 생깁니다. 그래서 위상정렬은 DAG에서만 가능합니다.

DFS 기반 위상정렬 코드

```
def topological_sort(graph):
    visited = set()
    result = []

    def dfs(node):
        visited.add(node)

        for nxt in graph[node]:
            if nxt not in visited:
                dfs(nxt)

        result.append(node)

    for node in graph:
        if node not in visited:
            dfs(node)

    result.reverse()
    return result
```

왜 마지막에 reverse를 하나

DFS에서 어떤 정점의 처리가 완전히 끝난 뒤 결과에 넣으면, 자연스럽게 "뒤쪽"에 쌓입니다. 그래서 마지막에 뒤집으면 선행 정점이 앞에 오게 됩니다.

사용 예시

```
graph = {
    "자료구조": ["알고리즘"],
    "알고리즘": ["그래프"],
    "그래프": ["위상정렬"],
    "위상정렬": [],
    "파이썬기초": ["자료구조"],
}

print(topological_sort(graph))
```

```
사용한 출력 예시:
['파이썬기초', '자료구조', '알고리즘', '그래프', '위상정렬']
```

위상정렬 결과는 하나만 있는 것이 아닐 수 있습니다. 중요한 것은 **선행 관계를 어기지 않는 순서**냐는 점입니다.

7. 한 파일에 모아서 돌려보는 전체 코드

```
from collections import deque

def bfs(graph, start):
    visited = set([start])
    queue = deque([start])
    order = []

    while queue:
        now = queue.popleft()
        order.append(now)

        for nxt in graph[now]:
            if nxt not in visited:
                visited.add(nxt)
                queue.append(nxt)

    return order

def dfs(graph, start, visited=None, order=None):
    if visited is None:
        visited = set()
    if order is None:
        order = []

    visited.add(start)
    order.append(start)

    for nxt in graph[start]:
        if nxt not in visited:
            dfs(graph, nxt, visited, order)

    return order

def topological_sort(graph):
    visited = set()
    result = []
```

```
    def visit(node):
        visited.add(node)

        for nxt in graph[node]:
            if nxt not in visited:
                visit(nxt)

        result.append(node)

    for node in graph:
        if node not in visited:
            visit(node)

    result.reverse()
    return result

graph1 = {
    1: [2, 3],
    2: [4],
    3: [4, 5],
    4: [],
    5: []
}

graph2 = {
    "마이선기초": ["자료구조"],
    "자료구조": ["알고리즘"],
    "알고리즘": ["그래프"],
    "그래프": ["위상정렬"],
    "위상정렬": []
}

print("BFS:", bfs(graph1, 1))
print("DFS:", dfs(graph1, 1))
print("위상정렬:", topological_sort(graph2))
```

실행하면 기대할 수 있는 결과

```
BFS: [1, 2, 3, 4, 5]
DFS: [1, 2, 4, 3, 5]
```

```
위상정렬: ['파이썬 기초', '자료구조', '알고리즘', '그래프', '위상정렬']
```

8. 재혁님이 구현할 때 자주 헷갈리는 부분

- 방문 처리를 늦게 하면 같은 정점이 여러 번 들어갈 수 있습니다.
- BFS는 큐, DFS는 재귀 또는 스택이라는 차이를 꼭 구분해야 합니다.
- 위상정렬은 아무 그래프에서나 되는 것이 아니라 DAG에서만 됩니다.
- 정점 번호가 1번부터 시작하는지 0번부터 시작하는지도 항상 확인해야 합니다.

특히 BFS에서 `queue.append()` 할 때 바로 방문 처리하는 습관을 들이면 중복 방문 실수를 줄이기 좋습니다.

9. 공부 순서 추천

1. 인접 리스트를 직접 손으로 한 번 써보기
2. BFS를 종이에 큐 상태까지 적어보기
3. DFS를 재귀 흐름으로 따라가 보기
4. 위상정렬에서 왜 reverse가 필요한지 말로 설명해보기
5. 마지막으로 직접 코드 타이핑해서 실행해보기

10. 마무리 한 번 더 정리

그래프

연결 관계를 저장하는 자료구조입니다.

BFS

가까운 정점부터 탐색하며, 큐를 씁니다.

DFS

한 방향으로 깊게 탐색하며, 재귀나 스택을 씁니다.

위상정렬

선행 관계를 지키는 순서를 만드는 방법이며 DAG에서만 가능합니다.

재혁님처럼 처음 배우는 단계에서는 "이론을 외우기"보다 **작은 그래프를 직접 그려 놓고 코드가 어떻게 움직이는지 확인하는 것**이 가장 중요합니다. 이 문서의 코드들을 직접 타이핑하고, 정점과 간선을 조금씩 바꿔보면 훨씬 빨리 익숙해질 수 있습니다.